Chapter 14: Indexes in MongoDB Atlas

Our Pets application now stores its documents in MongoDB Atlas, and it works. It does not yet tell us how much work the database does to find a document. With a handful of pets, every lookup feels instant. With a million pets, a lookup that reads every document is slow, and the delay grows with the data.

A database index is a separate, ordered structure that lets MongoDB find matching documents without reading the whole collection. (1) This chapter adds indexes to the Atlas Pets application, uses the MongoDB Shell to watch them work, and then reads the database's own account of how it answered a query.

We keep the same application. The Flask routes, templates, and integrity rules do not change. The only change to the application code is a small function in `database.py` that creates indexes when the program starts.

What an Index Is

A collection stores documents in no particular order that we can rely on. To find every pet whose `owner_id` matches Alex's identifier, MongoDB has two choices. It can examine each pet document and keep the ones that match, which is a **collection scan**. Or it can consult an index on `owner_id`, which keeps the values of that field in sorted order, each pointing to its document. That second choice is an **index scan**. (1)

A textbook's index works the same way. To find every page that mentions "replica set," you do not read the book. You look up the term and go to the listed pages.

Question	Without an index on the field	With an index on the field
How does MongoDB find matches?	Examines every document	Looks up the value in the index
How does the work grow with more data?	In proportion to collection size	Much more slowly
What does it cost?	Nothing extra stored	Extra storage, and extra work on every write

Every collection already has one index that MongoDB creates automatically, on the `_id` field. Looking up a pet by its identifier already uses it. (1) Indexes we add are for the other fields our queries use.

Which Pets Queries Need Help

An index can help when its fields support the query's filter, sort, or projection. Looking at `database.py`, the application's lookups fall into three groups.

Query in `database.py`	Filters on	Already indexed?
`get_pet`, `get_owner`, update and delete by identifier	`_id`	Yes, automatically
`get_pets`, `get_owners`	Nothing; they read everything	An index cannot help
`delete_owner` looks for any pet with this `owner_id`	`owner_id` on pets	No

That last lookup is the one we can improve today. Every time someone deletes an owner, the application asks whether any pet still refers to that owner. Without an index on `pets.owner_id`, MongoDB answers by reading pets until it finds a match or runs out of documents. When the owner has no pets, which is exactly when deletion succeeds, it reads the entire collection.

The application does not yet search by pet name, pet type, or owner name. We will still index those fields in this chapter, because a pet list that people can search by name and type is a natural next feature and because it gives us useful queries to examine in the shell.

Install the MongoDB Shell

The MongoDB Shell, `mongosh`, is a command-line program for talking to MongoDB. It connects to Atlas with the same connection string our Python code uses, and it lets us type queries and index commands one at a time and see the results at once. (2)

If you already have `mongosh`, check that it runs and skip ahead:

```
mongosh --version
```

These installation instructions are current as of this writing. Installation commands, package names, and supported operating systems change between releases. If a command here fails, use the official MongoDB Shell installation page, which is kept up to date: https://www.mongodb.com/docs/mongodb-shell/install/. (2)

You can work in a GitHub Codespace or on your own computer. Pick one of the paths below. Whichever you choose, finish with the check at the end of this section.

Option A: In a GitHub Codespace

A Codespace is a Linux machine in the cloud, usually running Ubuntu. Open its terminal and follow the Linux steps below. The Codespace keeps its software while it exists, but a rebuilt Codespace starts fresh and needs the installation repeated.

Option B: On Linux (Ubuntu or Debian)

MongoDB publishes `mongosh` through its own package repository. The repository line includes your Ubuntu release name, so it must match your system. Check your release first:

```
lsb_release -cs
```

Common answers are `focal` for Ubuntu 20.04, `jammy` for 22.04, and `noble` for 24.04. Then follow the current Linux instructions on the official installation page, which give the exact key and repository commands for your release. (2) The general sequence is: import MongoDB's public signing key, add MongoDB's repository to your package sources, update the package list, and install the `mongosh` package.

The repository commands depend on the operating system release. Use the official installation instructions for your release. (2)

If you cannot add a package repository, MongoDB also publishes `mongosh` as a downloadable archive that you unpack and add to your `PATH`. (3)

Option C: On macOS

The simplest route is Homebrew, a package manager for macOS. If you do not have Homebrew, install it from https://brew.sh/ first. Then run:

```
brew install mongosh
```

Homebrew's formula for `mongosh` is maintained separately from MongoDB's own instructions, so it can lag slightly behind a new release. (4)

Option D: On Windows

Windows offers two routes. If your computer has the `winget` package manager, open PowerShell and run:

```
winget install MongoDB.Shell
```

Otherwise, download the Windows installer from the MongoDB Shell download page, https://www.mongodb.com/try/download/shell, run it, and accept the defaults. (2) After installing, open a new PowerShell window so it picks up the updated `PATH`.

A Fallback If Node.js Is Installed

If you already have Node.js and its `npm` tool, you can run `mongosh` without installing it:

```
npx mongosh --version
```

The first run downloads the program. Use `npx mongosh` in place of `mongosh` in the rest of this chapter. This is convenient for a quick trial but slower to start than an installed copy. (3)

Check the Installation

Try This: Confirm that the shell installed and can be found:

```
mongosh --version
```

It prints a version number. If the terminal reports that the command is not found, open a new terminal window, then check that the installation finished and that its folder is on your `PATH`.

Connect mongosh to Atlas

`mongosh` uses the same connection string as the application. Use your own Atlas cluster and database user from Moving to MongoDB Atlas. The shared classroom deployment from that chapter was deleted.

Try This: In the terminal where you will run the shell, set the connection string and connect. Replace the value with your own:

```
export MONGODB_URI='mongodb+srv://USER:PASSWORD@YOUR-CLUSTER.mongodb.net/'
mongosh "$MONGODB_URI" --apiVersion 1
```

In Windows PowerShell:

```
$env:MONGODB_URI = 'mongodb+srv://USER:PASSWORD@YOUR-CLUSTER.mongodb.net/'
mongosh $env:MONGODB_URI --apiVersion 1
```

The `--apiVersion 1` option asks for the same Stable API our Python client requests with `ServerApi("1")`. (5) The prompt changes when the connection succeeds. Switch to the Pets database and list its collections:

```
use pets
show collections
```

You should see `owners` and `pets` if you created records in Moving to MongoDB Atlas. If the lists are empty, create an owner and a pet through the web application first, or insert a few documents in the shell as shown below.

If the connection fails, the troubleshooting table in Moving to MongoDB Atlas applies unchanged. A timeout usually means the IP access list; an authentication error usually means the username, password, or percent-encoding.

Make Enough Data to See a Difference

Indexes pay off on large collections. With four pets, a collection scan and an index scan both finish instantly, and the explain output will show tiny numbers either way. To see a real difference, we need many documents.

Try This: In `mongosh`, create one owner and then insert 50,000 pets that refer to it. This writes only to the Pets exercise database, so use a cluster where that is acceptable. Check available storage before loading; document and index sizes vary.

```
use pets
const ownerId = db.owners.insertOne({name: "Bulk Owner", city: "Kent", type_of_home:
        "house"}).insertedId
const types = ["dog", "cat", "mouse", "fish", "bird"]
const docs = []
for (let i = 0; i < 50000; i++) {
  docs.push({name: "pet" + i, type: types[i % 5], age: i % 20, owner_id: ownerId})
}
db.pets.insertMany(docs)
db.pets.countDocuments({})
```

The count should now be at least 50,000. Notice that every one of these pets refers to the same owner. That is unrealistic, and we will use it deliberately in a moment.

We also add a second owner with no pets, to stand in for an owner someone is about to delete:

```
const lonelyId = db.owners.insertOne({name: "No Pets", city: "Kent", type_of_home:
        "condo"}).insertedId
```

Reading a Query Plan with explain

The `explain` method asks MongoDB to describe how it would run a query. With the `"executionStats"` option, it also runs the query and reports what it actually did. (6) Three parts of the answer matter for us.

Part of the output	What it tells us
Stage names in the winning plan	`COLLSCAN` means the whole collection was read; `IXSCAN` means an index was used
`totalDocsExamined`	How many documents MongoDB read
`nReturned`	How many documents matched and were returned

A well-served query examines about as many documents as it returns. A query that examines 50,000 documents to return none is doing a lot of unnecessary work.

Try This: Ask whether the owner with no pets has any pets. This is the question `delete_owner` asks:

```
db.pets.find({owner_id: lonelyId}).explain("executionStats")
```

Find these three values in the long output: the stage name in `winningPlan`, `totalDocsExamined`, and `nReturned`. With no index on `owner_id`, expect a `COLLSCAN`, with `totalDocsExamined` equal to the size of the collection and `nReturned` equal to zero. MongoDB read every pet to learn that none matched.

> **Note on the output.** The layout of explain output differs between MongoDB server versions, and Atlas runs a current server version. If the stage names appear in a slightly different place than described here, look for `COLLSCAN` or `IXSCAN` anywhere in `winningPlan`. The supplied `explain-queries.py` searches for them this way for the same reason.

Create an Index

We create an index with `createIndex`, naming the field and a direction. A value of `1` keeps the index in ascending order; `-1` keeps it descending. For a single field, either direction serves equality lookups equally well. (7)

Try This: Create the index the application needs, and give it an explicit name:

```
db.pets.createIndex({owner_id: 1}, {name: "pets_owner_id"})
db.pets.getIndexes()
```

`getIndexes` lists every index on the collection. You should see the automatic `_id_` index and the new `pets_owner_id` index. Creating an index that already exists with the same definition does nothing, so the command is safe to repeat. (7)

Now repeat the same explain command:

```
db.pets.find({owner_id: lonelyId}).explain("executionStats")
```

The stage should now be `IXSCAN`, and `totalDocsExamined` should be zero. MongoDB looked up the value in the index, found no entries, and never read a pet document at all.

Compare it with a lookup that does match:

```
db.pets.find({owner_id: ownerId}).limit(1).explain("executionStats")
```

This uses the index too, and with `limit(1)` it stops after one match. Remove the `limit` and the query returns all 50,000 pets. An index does not make returning 50,000 documents cheap. It only avoids reading documents that do not match. We chose this data so that one owner has every pet, which is the worst case for an index on `owner_id`. In a realistic collection, each owner has a few pets and the index narrows 50,000 documents to a handful.

Indexes on Name and Type

The same pattern works for the other fields.

Try This: Create indexes on pet name, pet type, and owner name, and examine a query on each:

```
db.pets.createIndex({name: 1}, {name: "pets_name"})
db.pets.createIndex({type: 1}, {name: "pets_type"})
db.owners.createIndex({name: 1}, {name: "owners_name"})

db.pets.find({name: "pet49999"}).explain("executionStats")
db.pets.find({type: "dog"}).explain("executionStats")
db.owners.find({name: "No Pets"}).explain("executionStats")
```

The lookup by a specific pet name examines one document. The lookup by `type: "dog"` is different. One in five of our pets is a dog, so the index narrows 50,000 documents to 10,000, and it still returns 10,000 documents. The optimizer may choose an index scan or a collection scan. Check the actual winning plan. How much an index helps depends on how selective the query is, meaning how small a fraction of the collection it matches. (8)

Compound Indexes

A **compound index** covers more than one field. It serves queries that filter on the leading fields, in order. Suppose a future page lists one owner's pets of a given type:

```
db.pets.find({owner_id: ownerId, type: "dog"})
```

Try This: Create a compound index on `owner_id` and `type`, then look at the plan:

```
db.pets.createIndex({owner_id: 1, type: 1}, {name: "pets_owner_id_type"})
db.pets.find({owner_id: ownerId, type: "dog"}).explain("executionStats")
```

A compound index can serve a query on its first field alone, so `pets_owner_id_type` can also answer `{owner_id: ...}` queries. It cannot efficiently serve a query on only `type`, because `type` is not the leading field. The order of the fields in a compound index matters. (9)

That observation raises a design question. Once the compound index exists, the single-field `pets_owner_id` index is largely redundant, because the compound index covers the same lookups. Every index we keep has a cost, as the next section describes. The application's startup code in this chapter keeps the simple single-field indexes so each one is easy to see. A production design would likely replace `pets_owner_id` with the compound index.

Unique Indexes

A **unique index** rejects a second document with the same value in the indexed field. (10) It is how MongoDB enforces uniqueness, the role that a `UNIQUE` constraint played in the relational version.

Our Pets data does not call for one. Two owners may share a name, as we saw in A Web Application with Mongita, and so may two pets. A field such as an email address on a user account would be a typical candidate. Be careful with documents that lack the field: a unique index treats a missing value as a value, so two documents without the field collide. (10)

What Indexes Cost

An index is not free. Every insert must add an entry to each index on the collection. Every update to an indexed field must change that index. Every delete must remove entries. Indexes also take storage, and MongoDB works best when the indexes it uses most often fit in memory. (1)

Operation	Effect of adding an index
Read that filters on the indexed field	Can be faster; check the winning plan
Read on an unindexed field	No change
Insert, update of an indexed field, or delete	Slower; each index must be maintained
Storage	More used

The practical rule is to index the fields your queries actually filter and sort on, and no more. Our `name` and `type` indexes are justified here as teaching examples. In a real application, add an index when a real query needs it, and use explain to confirm that the query uses it.

Try This: See the storage cost for yourself:

```
db.pets.stats().indexSizes
```

Each index appears with its size in bytes. Compare the sizes with each other and note which index is largest.

Add Indexes to the Application

The shell is for experiments. The application needs the indexes to exist whenever it runs against a fresh database, so we create them in `database.py` as part of startup.

The supplied `database.py` begins with a table of the indexes we want:

```
INDEXES = [
    ("pets", "owner_id", "pets_owner_id", {}),
    ("pets", "name", "pets_name", {}),
    ("pets", "type", "pets_type", {}),
    ("owners", "name", "owners_name", {}),
]
```

A new function walks the table and creates each index, and `setup_database()` calls it after checking read access:

```
def ensure_indexes():
    collections = {"owners": owners_collection, "pets": pets_collection}
    created = []
    for collection_name, field, index_name, options in INDEXES:
        collections[collection_name].create_index(
            [(field, 1)], name=index_name, **options
        )
        created.append(index_name)
    return created
```

PyMongo's `create_index` takes a list of field and direction pairs and an optional name. It does nothing if an identical index already exists, so the application can call it on every start. (11) That is why startup does not need to ask whether the indexes exist first.

This function is meant for a real MongoDB deployment such as Atlas, and it does not catch errors, so a real problem such as a missing permission is visible. The application's unit tests use Mongita's in-memory store, which cannot create named indexes, so they never call it. The `mongosh` exercises and `explain_queries.py` are how we check the indexes in Atlas.

Creating an index writes to the database, so the Atlas database user needs permission to do it. The `readWrite` role from Moving to MongoDB Atlas includes index creation on the `pets` database. (12) A user with read-only access could not start this version of the application.

Try This: From the `chapter-14-mongo-indexes/code` folder, with `MONGODB_URI` set and the dependencies installed as in Moving to MongoDB Atlas, start the application:

```
flask --app app run --debug
```

Back in `mongosh`, list the indexes again with `db.pets.getIndexes()`. Every index named in the table should be present. Drop one in the shell with `db.pets.dropIndex("pets_type")`, restart the application, and list the indexes once more. The application recreates it.

Reading Plans from Python

The shell is convenient for experiments, and Python is where the application lives. PyMongo can request the same explain output from a cursor.

Try This: Run the supplied script:

```
python explain-queries.py
```

It prints the indexes on each collection, and then, for each of the application's main lookups, the plan stages, whether the query used an index, and the documents examined and returned. It includes one query on `age`, which has no index, so you can see a collection scan in the same report. The script reads data and creates the application's indexes through `setup_database()`, but it does not insert, update, or delete any documents.

The code that asks for the plan is one line:

```
result = collection.find(query).explain()
```

The rest of the script finds the stage names in the result and prints them in plain terms. (6) We search for the stage names instead of relying on one fixed path, because the layout of explain output can differ between server versions.

Performance Experiments

Explain tells us what the database did. It does not tell us how long it took on our network. Atlas queries cross the internet, so timing from our computer includes network delay that has nothing to do with indexes. Document counts are a more reliable measure than elapsed time for comparing a scan with an index scan.

If you do measure time, the explain output includes `executionTimeMillis`, which is the server's own time for the query, separate from network delay. (6) Run each query several times, because the first run may be slower while the server loads data into memory.

Practice

1. Before creating any indexes, run explain with `"executionStats"` on `{owner_id: lonelyId}`. Record the stage, `totalDocsExamined`, and `nReturned`. Create the `pets_owner_id` index and repeat. Explain what changed and what did not.

2. Run explain on `{type: "dog"}` and on `{name: "pet49999"}`. Either may use an index. Why does one examine far more documents than the other?
3. Create the compound index on `owner_id` and `type`. Run explain on a query on `type` alone. Does it use the compound index? Explain why or why not.
4. Use `db.pets.stats().indexSizes` to compare index sizes. Insert 10,000 more pets and compare the sizes again. Which index grew the most?
5. Run `explain_queries.py`. Find the one query that shows a collection scan, and explain why none of the indexes can help it.

Considerations

- The application already enforces its integrity rules in `database.py`. Does adding indexes change any of those rules, or only how quickly they are checked?
- Which of the four indexes in `database.py` would you drop first if write speed mattered more than read speed, and why?
- If most owners have a few pets, why is an index on `owner_id` more valuable than one on `type`?
- Why can a compound index on `owner_id` and `type` replace the single-field index on `owner_id`, but not the one on `type`?
- Explain output depends on the data and on the server. What should you do before trusting a plan you saw on a small test collection?

Database indexes also support relational systems; the same storage and write tradeoffs apply there. (13)

References

1. MongoDB. “Indexes.” MongoDB Manual. https://www.mongodb.com/docs/manual/indexes/
2. MongoDB. “Install mongosh.” MongoDB Shell documentation. https://www.mongodb.com/docs/mongodb-shell/install/
3. MongoDB. “mongosh: The MongoDB Shell.” GitHub repository. https://github.com/mongodb-js/mongosh
4. Homebrew. “mongosh.” Homebrew Formulae. https://formulae.brew.sh/formula/mongosh
5. MongoDB. “Stable API.” MongoDB Manual. https://www.mongodb.com/docs/manual/reference/stable-api/
6. MongoDB. “Analyze Query Performance.” MongoDB Manual. https://www.mongodb.com/docs/manual/tutorial/analyze-query-plan/
7. MongoDB. “createIndex.” MongoDB Manual. https://www.mongodb.com/docs/manual/reference/method/db.collection.createIndex/
8. MongoDB. “Query Selectivity.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/query-optimization/
9. MongoDB. “Compound Indexes.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/indexes/index-types/index-compound/

10. MongoDB. “Unique Indexes.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/indexes/index-properties/

11. MongoDB. “Indexes.” PyMongo documentation. https://www.mongodb.com/docs/languages/python/pymongo-driver/current/indexes/

12. MongoDB. “Built-In Roles.” MongoDB Manual. https://www.mongodb.com/docs/manual/reference/built-in-roles/

13. Wikipedia contributors. “Database index.” https://en.wikipedia.org/wiki/Database_index